

VideoMind: Semantic Search Over Surveillance Video

System architecture, design rationale, and requirements

Problem Understanding

Surveillance footage on an NVR has no index beyond the clock, so finding an incident means guessing a time window and scrubbing through it. An operator who knows what happened — “a man in a yellow shirt left a bag near the checkout” — first has to translate that into when it happened, because nothing in the pipeline has ever described the content.

Surveillance footage is also the hardest input for generic video AI: a fixed camera produces no shot cuts, long stretches carry no speech, and the events that matter are brief and visually ordinary. Systems built for edited media either collapse the footage into one giant chunk or fragment it into thousands of meaningless ones.

The task is to build a semantic layer over footage that offers almost no natural boundaries, and expose it behind a single prompt box that returns the moment itself — timestamped and playable.

How We Solve It

VideoMind runs a chunk → analyze → index → aggregate → search/ask pipeline, and every stage is plural because no single setting survives real footage.

- Chunking — three modes: preset (weighted fusion of speaker change, silence, hard cuts, and semantic drift), weights, and interval for footage with no structure at all. Weights renormalise over whichever signals actually fired, so unbroken CCTV still chunks on semantic drift instead of collapsing into one block.
- Multi-granularity indexing — vectors are keyed on chunk configuration, so one video can hold a coarse and a fine chunking side by side, and a query picks whichever granularity it needs.
- Analyzers — selected per upload (description, people, object detection, OCR, transcript/diarization) since their costs differ widely. Each has its own frame sampler: an adaptive similarity threshold with an absolute ceiling, cheap grab()/retrieve() reads instead of seeking, and detectors like YOLO/OCR gating access so empty frames never reach a VLM.
- Retrieval — each chunk is one vector-database point with several named vectors (combined, description, people, actions, objects), so an appearance query matches a short person description instead of competing with an entire record. Search adds structured filters and selectable response detail; question answering routes to video-level aggregates — entities for a person, novelty for “what looks unusual,” stats for “how busy was it.”

The operator types a description and gets ranked moments with mm:ss timecodes; the media endpoint supports range requests, so clicking a result seeks straight to the incident.

VideoMind System Architecture

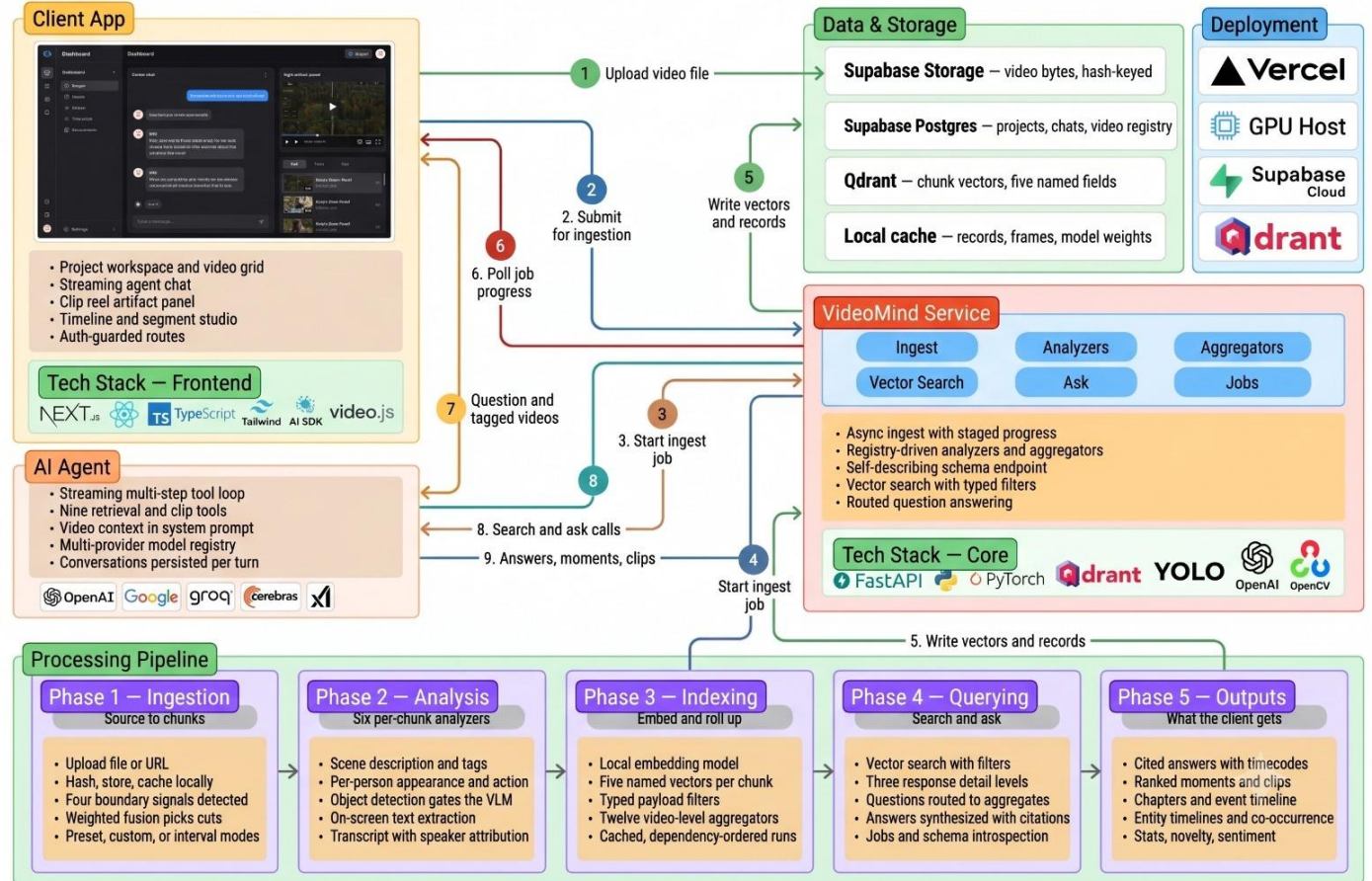


Figure 1. VideoMind system architecture.

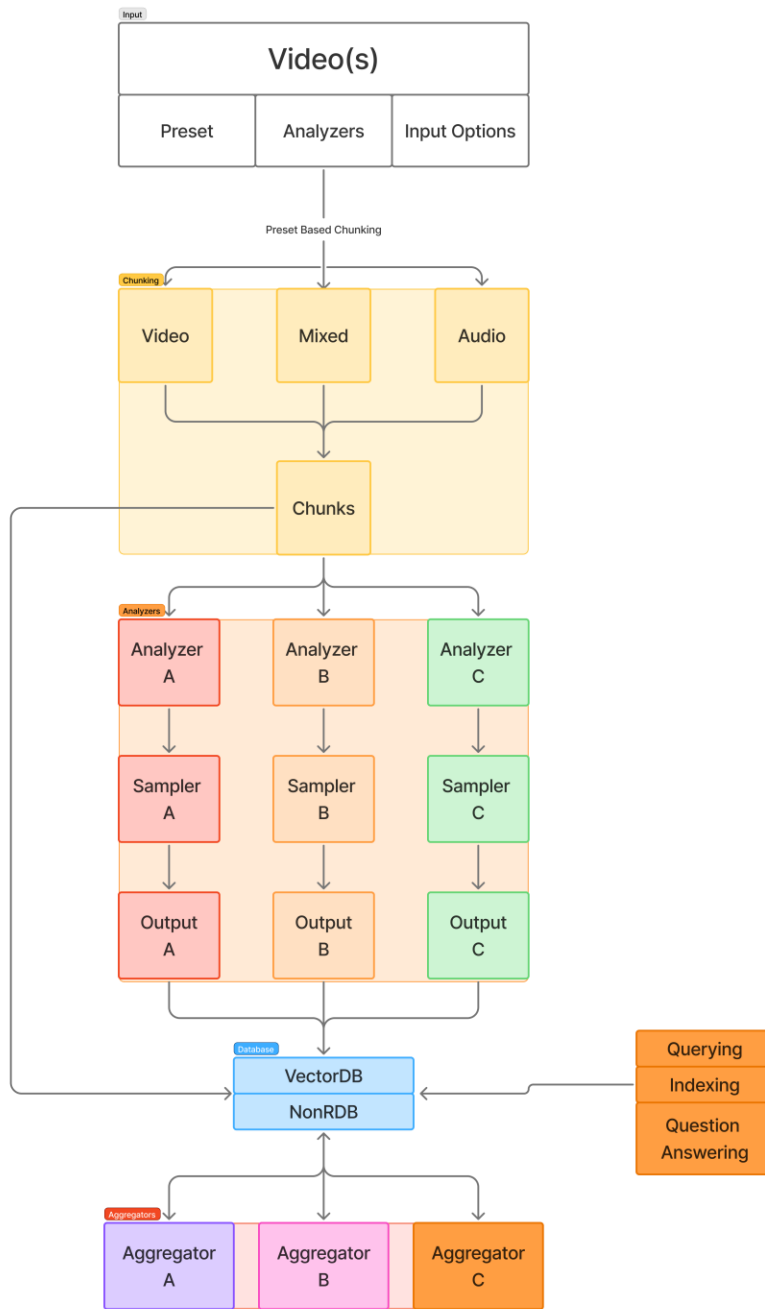


Figure 2. Chunking, analyzer, and aggregator processing flow.

Functional Requirements

- Ingest a recorded video file with a per-upload choice of chunking mode (preset / weights / interval, with min-max duration bounds) and analyzer set, rejecting invalid combinations such as transcript + diarization.
- Run ingestion as a background job with a job id that reports stage and progress (chunking → analyzing → indexing → complete).
- Analyze, embed, and store each chunk so it is retrievable by natural-language text alone. Identify videos by content hash so re-uploads are recognised, and treat re-chunking as adding a parallel index rather than overwriting one.
- Support free-text search over a chosen analyzer and vector field, with structured filters (time range, objects, tags, people count, speakers, video ids), a score threshold, and selectable response detail.
- Route questions to the video-level aggregates that can answer them, and respond with citations rather than raw segments.

- Return start/end and a human-readable timecode with every result, and stream source video with HTTP range support so a result can be played from its exact moment.
- Make aggregates (summary, chapters, events, entities, timelines, stats, novelty, NER, sentiment) re-runnable from saved records without re-analysing the video, and invalidate them when the analyzer set changes.

Non-Functional Requirements

- Cost control is the binding constraint: gate and deduplicate frames before analysis, select analyzers per upload, cache and reuse aggregate results unless a re-run is forced, and support re-indexing from stored records with no paid calls.
- Responses must be size-aware: minimal detail should stay around a few hundred tokens against tens of thousands for full detail.
- The pipeline must stay robust on degenerate input — no cuts, no speech, or a static camera — producing usable chunks and descriptions rather than failing or collapsing.
- Everything except VLM calls and answer synthesis must run locally on a single consumer GPU (developed on an 8 GB RTX 4060), constraining memory and model choices.
- Extensibility is first-class: adding an analyzer or aggregator should touch one module plus a registry line, with no changes to ingest, the store, or the API; a new store filter should be reachable over HTTP without extra plumbing.
- Failures must degrade rather than abort — an aggregator whose analyzer is absent is skipped, not failed — and errors must be explicit, with unknown filters or analyzers rejected with a suggestion instead of silently ignored.
- All runtime state must live under one path-configurable directory, so a reset is a single delete and a second instance can run in isolation.